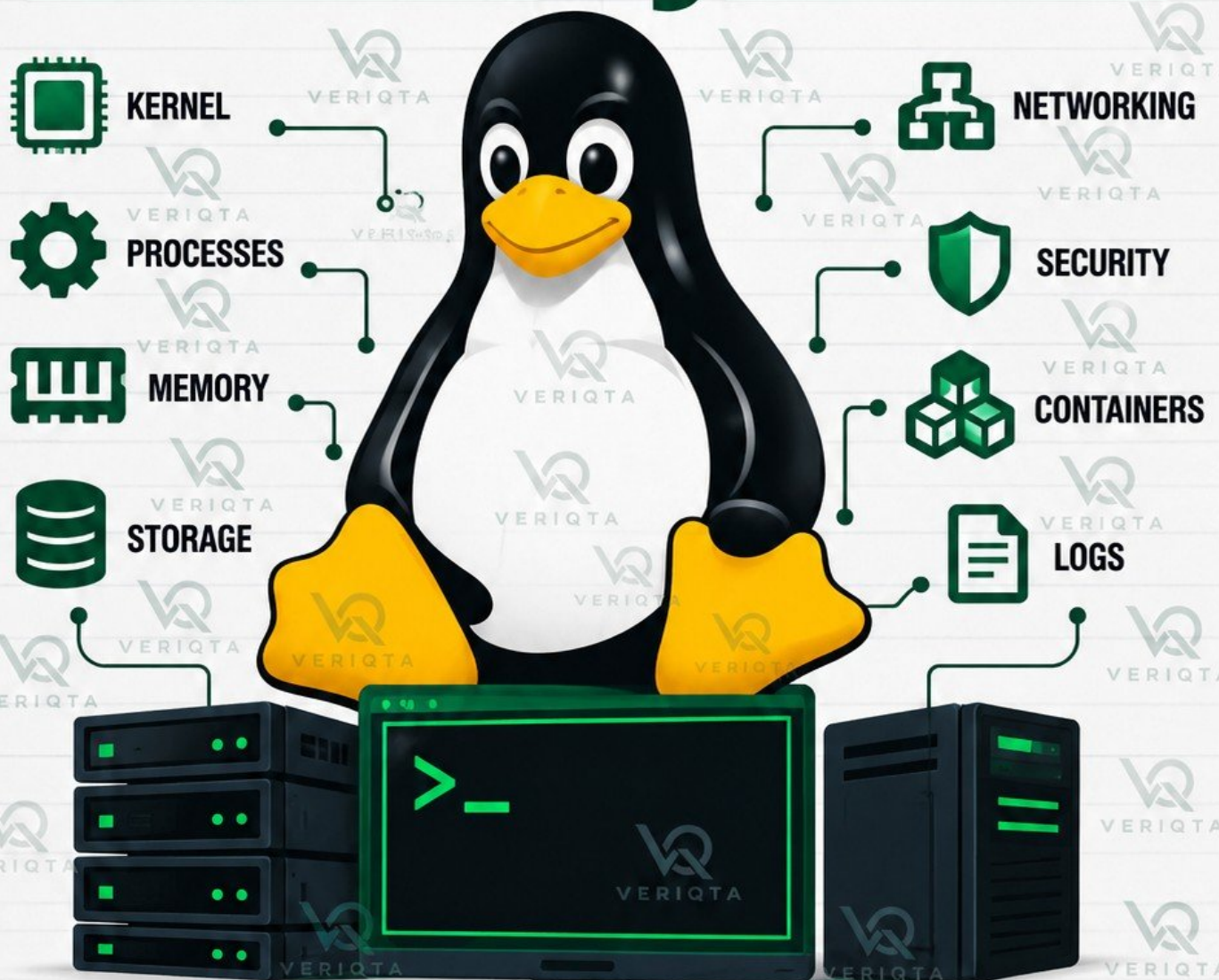


Anatomy of a Linux System



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Anatomy of a Linux System

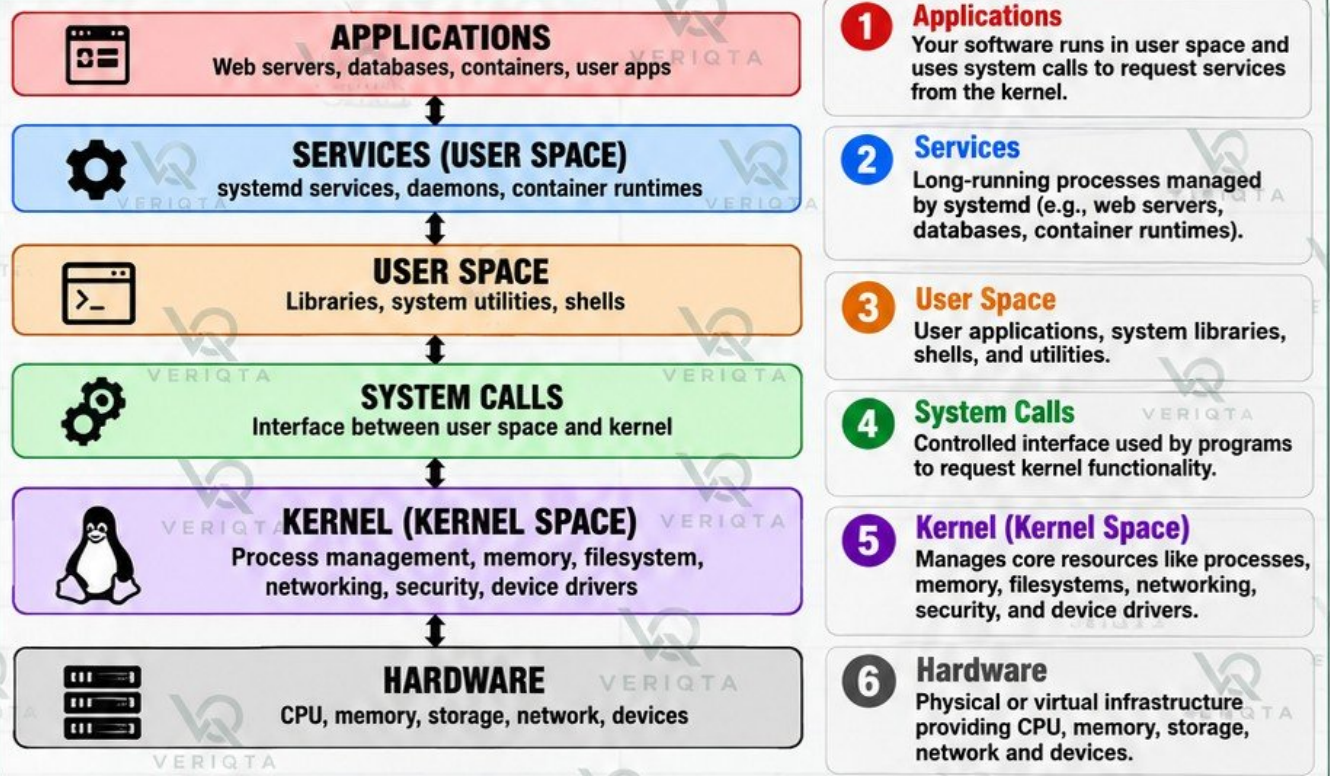
1. OVERVIEW

A Linux system is made up of multiple layers working together, from the underlying **hardware** to the **applications** you run. Each layer has a specific role and interacts through well-defined interfaces.

KEY TAKEAWAY

Linux is not just a collection of commands. It is a layered system where components communicate through the kernel using **system calls**.

2. LINUX SYSTEM ARCHITECTURE



3. KERNEL SPACE VS USER SPACE

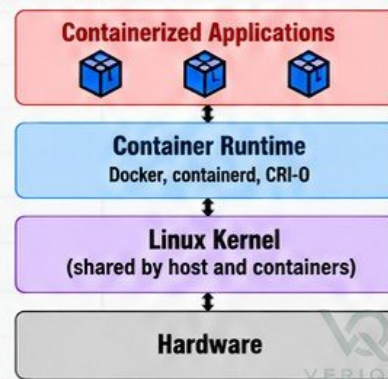
KERNEL SPACE

- Runs the kernel code
- Full access to hardware
- Higher privilege level
- Handles core system functions
- Example: process scheduler, memory management

USER SPACE

- Runs applications and services
- Limited privileges
- Uses system calls to access kernel functionality
- Isolated for stability and security
- Example: web server, database, shell

4. WHERE CONTAINERS FIT



5. CONTROL PATHS VS DATA PATHS

CONTROL PATH

- System calls
- Process and service management
- Configuration changes
- Scheduling and resource allocation
- Examples: systemd, API calls, admin commands

DATA PATH

- Actual data flow through the system
- File reads/writes
- Network traffic
- I/O between applications and devices
- Examples: HTTP requests, database queries, disk I/O, packet flow

6. SENIOR ENGINEER INSIGHT

When troubleshooting a Linux system, always start by identifying which layer owns the failure. Is it hardware, kernel, user space, a service, or the application? Correctly identifying the layer helps you focus on the right evidence and avoid wasting time.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



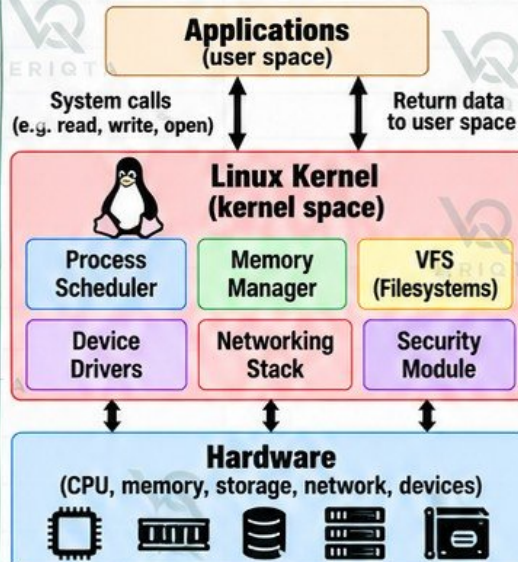
@veriqta

The Linux Kernel

1. WHAT THE KERNEL ACTUALLY CONTROLS

The Linux kernel is the core of the operating system. It manages **hardware resources**, provides core **system services**, and enforces isolation, **security**, and **access control**. It acts as a bridge between hardware and user space applications through system calls.

2. KERNEL ARCHITECTURE IN CONTEXT



3. KEY RESPONSIBILITIES

- 1 Process scheduling
- 2 Virtual memory
- 3 Filesystems
- 4 Device management
- 5 Networking stack
- 6 Security enforcement
- 7 Kernel modules
- 8 /proc, /sys and dmesg
- 9 Reliable and stable access to hardware resources

4. PROCESS SCHEDULING



- Decides which process runs on the CPU and for how long.
- Manages CPU time, priorities and fairness.
- Supports multiple scheduling algorithms (e.g. CFS).
- Affects application performance and system responsiveness.

5. VIRTUAL MEMORY



- Provides each process with its own virtual address space.
- Handles memory allocation, page tables and paging.
- Manages swap and memory reclaim.
- Protects processes from accessing each other's memory.

6. FILESYSTEMS



- Manages files and directories through the Virtual File System (VFS).
- Supports multiple filesystems (e.g. ext4, XFS, Btrfs).
- Handles permissions, metadata and caching.
- Provides a unified interface for applications to access storage devices.

7. DEVICE MANAGEMENT



- Controls hardware devices through device drivers.
- Manages device discovery, initialization and I/O.
- Handles device files (e.g. /dev).
- Supports hot-plug and hardware events (udev).

8. NETWORKING STACK



- Implements network protocols (TCP, UDP, IP, ICMP, etc).
- Manages network interfaces and routing.
- Handles packet processing, firewalling (netfilter/iptables/nft).
- Provides sockets for applications to communicate over the network.

9. SECURITY ENFORCEMENT



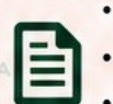
- Enforces process isolation, permissions and capabilities.
- Implements security modules (Linux Security Modules - LSM) like SELinux or AppArmor.
- Controls access to sensitive resources.
- Validates system calls and prevents unauthorized actions.

10. KERNEL MODULES



- Extend kernel functionality without rebooting.
- Provide support for hardware drivers, filesystems and features.
- Can be loaded or unloaded at runtime (modprobe, insmod).
- **Production Risk:** a faulty module can crash the kernel or cause system instability.

11. /proc, /sys AND dmesg



- /proc – provides runtime information about processes, memory, CPU, etc.
- /sys – exposes kernel and device information (drivers, hardware, config).
- dmesg – shows kernel messages, errors and hardware events.
- Essential for troubleshooting and understanding kernel behavior.

12. PRODUCTION RISK OF KERNEL-LEVEL FAILURES



- Kernel panics can take down the entire system.
- Faulty drivers or modules can cause crashes or data corruption.
- Memory management issues can lead to OOM and service outages.
- Network stack bugs can cause connectivity loss.
- Kernel vulnerabilities can be exploited for full system compromise.

SENIOR ENGINEER INSIGHT



When troubleshooting a Linux system, always start by identifying which layer owns the failure. Is it the application, user space, a system call, the kernel, or the hardware? Correctly identifying the layer helps you focus on the right evidence and avoid wasting time.



@veriqta



@veriqta



@veriqta



@veriqta



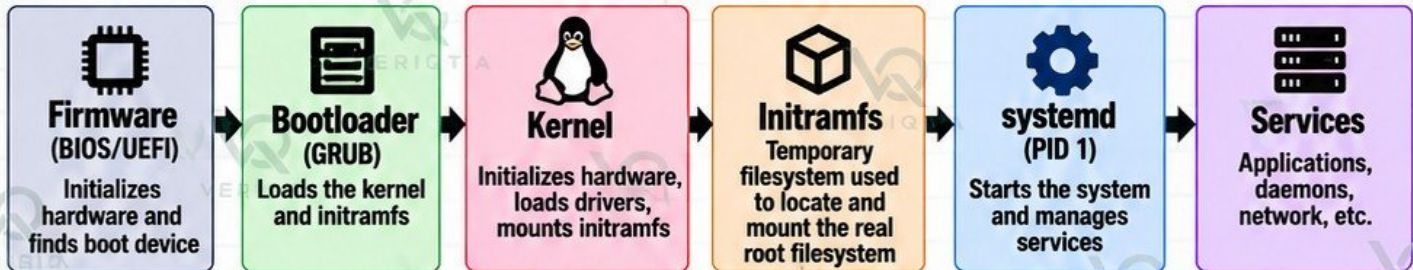
@veriqta



@veriqta

Linux Boot Anatomy

From power on to a fully running system with services



1. KERNEL INITIALIZATION



- Decompresses and initializes the kernel.
- Initializes CPU, memory, devices and core subsystems.
- Loads built-in and required kernel modules.
- Mounts the initramfs (initial RAM filesystem).
- Prepares the system to switch to the real root filesystem.

2. ROOT FILESYSTEM DISCOVERY



- Kernel and initramfs locate the real root filesystem (e.g. /dev/sda1, LVM, RAID, NVMe, or network storage).
- Uses kernels parameters, initramfs scripts and drivers.
- If the root filesystem cannot be found or mounted, the boot process stops or drops into an emergency shell.

3. SYSTEMD TARGETS



- systemd uses targets to define the system state.
- Common targets:
 - **initrd.target** (early boot)
 - **multi-user.target** (normal)
 - **graphical.target** (with GUI)
 - **rescue.target** (limited shell)
 - **emergency.target** (minimal shell)

4. BOOT DEPENDENCIES



- Services are started based on dependency relationships.
- Units can define:
 - **Requires=** (must be started)
 - **Wants=** (optional)
 - **After=** (start order)
 - **Before=** (start order)
 - **Conflicts=** (cannot run together)
- Incorrect or missing dependencies can delay or prevent critical services from starting.

5. FAILED UNITS AND DEGRADED BOOT



- The system can complete boot even if some services fail.
- Failed units may cause partial functionality or critical workloads to be unavailable.
- Use **systemctl --failed** to identify failed services.
- Check logs to understand the reason for failure.
- A degraded boot is often worse than a clear failure because the system appears "up" but key services are down.

6. USEFUL COMMANDS AND EVIDENCE

View boot logs
`journalctl -b`

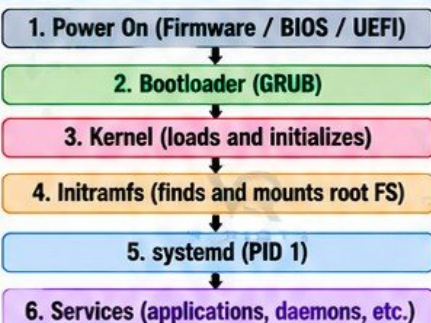
List failed units
`systemctl --failed`

Analyze boot time
`systemd-analyze`

Detailed boot timeline
`systemd-analyze blame`

View service status
`systemctl status <service>`

7. LINUX BOOT FLOW (SIMPLIFIED)



8. EXAMPLE BOOT TIMELINE

```
$ systemd-analyze
Startup finished in 2.341s (firmware) + 1.203s (loader) + 4.567s (kernel) + 1.890s (initrd) + 7.321s (userspace) = 17.322s
```

```
$ systemd-analyze blame
7.321s network-online.target
3.210s docker.service
1.405s kubelet.service
980ms containerd.service
620ms ssh.service
```

9. COMMON BOOT ISSUES



- Kernel panic during initialization
- Root filesystem not found
- Missing or wrong drivers
- Failed or hanging services
- Dependency failures
- Slow boot due to misconfigured units or network timeouts
- System boots but critical workloads (e.g. Docker, Kubernetes, web server) are not running

10. PRODUCTION FAILURE SCENARIO



The system boots successfully and shows as "up", but critical workloads never become available (e.g. container runtime, application services, database). This is often caused by failed units, incorrect dependencies, or services waiting on network or storage that never becomes ready.

WHAT TO DO

- Check failed units: `systemctl --failed`
- Review boot logs: `journalctl -b`
- Analyze boot time: `systemd-analyze`
- Check service dependencies
- Verify network, storage and mounts
- Check application-specific logs



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

systemd and Service Lifecycle

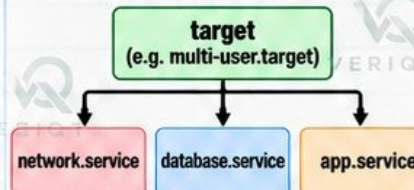
1. PID 1 RESPONSIBILITIES



- systemd runs as PID 1 and is the first process started by the kernel.
- Initializes the userspace.
- Manages system services and their dependencies.
- Handles targets (runlevels).
- Manages mounts, devices, networking and sockets.
- Monitors service health and restarts them when configured.
- Handles shutdown and reboot.

2. UNITS AND DEPENDENCY RELATIONSHIPS

- Services, sockets, mounts, devices, timers, and more are managed as units.
- Units define how services start, stop and relate to each other.
- Dependencies ensure services start in the correct order.



3. REQUIRES, WANTS, AFTER, BEFORE

Requires=

- Strong dependency.
- If the required unit fails, this unit also fails.

Wants=

- Weak dependency.
- This unit still starts even if the wanted unit fails.

After=

- Start this unit after the specified unit.
- Does not pull in the unit.

Before=

- Start this unit before the specified unit.
- Used to control startup order.

4. SERVICE STARTUP AND RESTART BEHAVIOR

- systemd starts services based on their dependency relationships.
- Service type (simple, forking, oneshot, notify) defines how systemd considers the service "started".
- Restart behavior can be configured using Restart=.
- systemd can automatically restart failed services.

Example Service Configuration

```

[Service]
ExecStart=/usr/bin/myapp      # Command to start
Restart=on-failure            # Restart on failure
RestartSec=5                  # Wait 5 seconds
TimeoutStartSec=30            # Start timeout
Type=simple                   # Service type
  
```

5. FAILED SERVICE STATES



- A service can be in failed state if it exits with a non-zero status or cannot be started.
- Common reasons:
 - Missing dependencies
 - Configuration errors
 - Permission issues
 - Resource limitations
 - Application crash
 - Timeout during startup

Check the status and logs to understand the exact reason for the failure.

6. RESTART LOOPS AND DEPENDENCY FAILURES



- A service may enter a restart loop if it keeps failing and restarted automatically.
- This can hide the real problem and consume resources.
- Dependency failures can prevent a service from starting, even if the service configuration is correct.
- Use `systemctl --failed` to check failed units.
- Check logs to identify the actual root cause.

7. USEFUL COMMANDS

```

systemctl status nginx      # Check service status
journalctl -u nginx         # View service logs
systemctl --failed          # List failed units
systemctl restart nginx     # Restart a service
systemctl enable nginx      # Enable at boot
systemctl disable nginx     # Disable at boot
systemctl list-dependencies nginx # Show dependencies
  
```

8. ARCHITECTURE TRADE-OFF



Automatic Restart (Pros)

- Improves availability.
- Recovers from transient failures.
- Reduces manual intervention.
- Useful for stable and well-behaved applications.



Automatic Restart (Cons)

- Can hide persistent failures.
- May cause restart loops.
- Can consume resources.
- Delays detection of real issues if not monitored properly.



Senior Engineer Insight

Automatic restart is useful for resilience, but it can also hide persistent failures. Always monitor service health, check restart counts, and investigate the root cause instead of relying only on automatic restarts.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



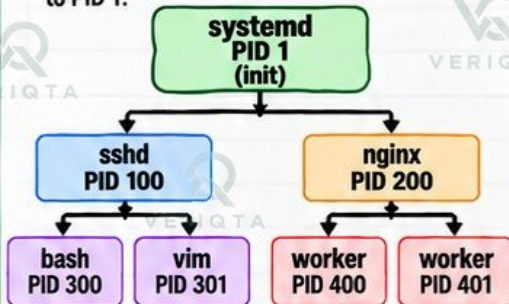
@veriqta

Process Anatomy

Understanding how Linux processes work from creation to termination

1. PID, PPID AND PROCESS HIERARCHY

- Every process has a unique Process ID (PID).
- Each process has a Parent Process ID (PPID).
- Processes form a tree, with init/systemd (PID 1) at the root.
- Children inherit resources and environment from the parent.
- When a parent exits, children are re-parented to PID 1.



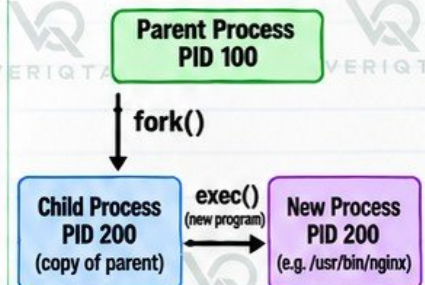
2. PROCESS STATES

A process moves through different states during its lifecycle.

State	Code	Meaning
Running	R	Runnable or currently running
Sleeping	S	Waiting for an event (interruptible)
Uninterruptible Sleep	D	Waiting for I/O (cannot be interrupted)
Stopped	T	Stopped (e.g. by signal)
Zombie	Z	Process finished but not reaped by parent
Dead	X	Terminated

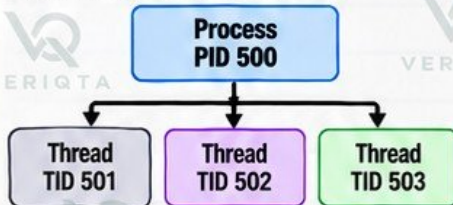
3. FORK AND EXEC

- `fork()` creates a new process by copying the parent.
- The child gets a new PID.
- `exec()` replaces the child process image with a new program.
- This is how shells start new programs.



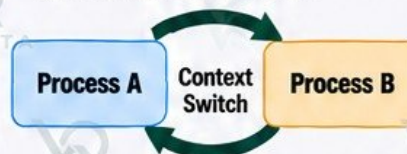
4. THREADS

- A process can have multiple threads of execution.
- Threads share the same memory space, file descriptors and resources.
- Each thread has a unique Thread ID (TID).
- Threads are lighter weight than processes and allow concurrent execution.



5. CONTEXT SWITCHING

- The CPU switches between processes and threads very quickly.
- The kernel saves the current context (registers, state) and loads the next one.
- Allows multiple processes to share CPU time (time slicing).
- High context switching can reduce performance if too frequent.



- Use `vmstat`, `pidstat` or `perf` to check context switch rates.

6. SIGNALS

- Signals are used to notify a process about an event.
- Common signals:

Signal	Number	Meaning
SIGHUP	1	Reload configuration
SIGINT	2	Interrupt (Ctrl+C)
SIGTERM	15	Graceful termination
SIGKILL	9	Force kill (cannot ignore)
SIGSTOP	19	Stop process
SIGCONT	18	Continue process

- Use `kill -l` to see all signals.
- Use `kill <signal> <PID>` to send a signal.

7. ZOMBIE AND ORPHAN PROCESSES



Zombie Process

- Process has finished execution but parent has not read its exit status.
- Remains in Z (zombie) state.
- Consumes a PID entry.
- Fixed when parent calls `wait()`.



Orphan Process

- Parent process exits before the child.
- Child is re-parented to PID 1 (systemd).
- Continues running in the background.
- Common for long-running daemons and services.

8. /proc/<PID>

- The `/proc` filesystem exposes runtime information about processes.
- Useful for debugging and inspection.

```

/proc/1234/
├── cmdline # Command line
├── cwd     # Current working dir
├── environ # Environment variables
├── fd/     # Open file descriptors
├── maps   # Memory mappings
├── status # Process status
└── threads/ # Thread information
  
```

9. USEFUL COMMANDS

```

ps aux          # List processes
pstree -p       # Show process tree
top             # Real-time process view
htop           # Interactive (if installed)
ps -eo pid,ppid,cmd # Custom output
pstree -ap <PID> # Show tree for a PID
  
```

10. DEBUGGING INSIGHT



High CPU does not automatically identify the root cause.

High CPU usage could be due to application logic, excessive threads, busy loops, lock contention, high context switching, or the kernel spending time in interrupts or I/O handling. Always investigate: which process, what it is doing, and why.

INVESTIGATION STEPS

- Identify the process (`top`, `ps`, `pidstat`)
- Check what it is doing (`strace`, `perf`, stack traces)
- Look at system context (CPU, I/O wait, interrupts)
- Correlate with recent changes and logs



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

CPU Scheduling and System Load

Understanding how Linux schedules work, measures load and uses CPU resources

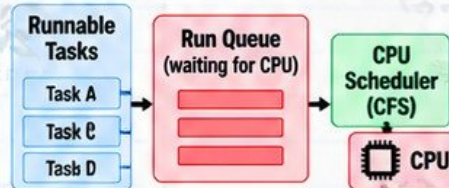
1. CPU SCHEDULING OVERVIEW



- The Linux scheduler decides which runnable task runs on each CPU and for how long.
- Uses algorithms like CFS (Completely Fair Scheduler).
- Shares CPU time fairly among tasks based on priority, weights and scheduling policies.
- Handles task states (runnable, sleeping, blocked) and performs context switching.

2. RUNNABLE TASKS AND RUN QUEUE

- Runnable tasks are tasks ready to run but waiting for CPU.
- Each CPU has a run queue.
- The scheduler picks the next task from the run queue based on priority and fairness.
- If the run queue is long, tasks wait longer, which increases latency.



3. CPU UTILIZATION VS SATURATION

- CPU Utilization:** percentage of time CPU is doing useful work (user + system).
- High utilization does not always mean saturation.
- CPU Saturation:** run queue keeps growing, tasks wait, latency increases.
- You can have 100% utilization but not be saturated if tasks are mostly I/O bound.

4. LOAD AVERAGE

- Load average shows the average number of runnable or uninterruptible tasks over 1, 5 and 15 minutes.
- It is NOT a percentage.
- Compare load average to the number of CPU cores to understand pressure.

\$ uptime

10:24:01 up 3 days, 2 users,
load average: 2.15, 1.98, 1.76

1.00 on 4 cores	Normal
4.00 on 4 cores	High load
8.00 on 4 cores	System is saturated

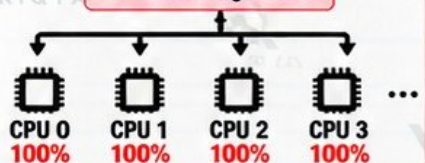
5. CPU TIME CATEGORIES

Type	Meaning	What it indicates
user	Time running user-space code	Application processing
system	Time running kernel code	Kernel and system calls
idle	CPU is idle	No work to do
iowait	Waiting for I/O to complete	Storage or network I/O bottleneck
steal	Time stolen by hypervisor (VM)	Contention on virtualized host

6. MULTI-CORE INTERPRETATION

- Modern servers have multiple CPU cores and threads.
- Load average and CPU usage must be interpreted per core.
- Example: On an 8-core system, a load average of 8 means high pressure.
- Use per-CPU metrics to identify which cores are busy.

Load Average 8.00



7. CONTEXT SWITCHES

- A context switch happens when the CPU switches from one task to another.
- High context switch rates can indicate too many runnable tasks, I/O waits, or inefficient applications.

\$ vmstat 1 5

```

procs -----memory-----swap--
r  b  swpd   free   buff   cache   si   so
2  0    0  521420  33412  682144  0  0
5  1    0  518320  33420  683024  0  0
7  3    0  510212  33480  686112  0  0
  
```

r = runnable tasks (run queue)
b = blocked tasks (usually I/O)

8. USEFUL COMMANDS

\$ uptime # load average and uptime

\$ vmstat 1 # runnable, blocked, memory,
context switches, CPU time

\$ mpstat -P ALL 1 # per-core CPU usage

\$ pidstat -u 1 # per-process CPU usage

\$ top # real-time process view

\$ htop # interactive process view

9. INTERPRETING CPU METRICS

- High user time → application is busy running code.
- High system time → heavy kernel activity (system calls, interrupts, networking).
- High iowait time → storage or network I/O bottleneck.
- High steal time → VM is contending for CPU on the host.
- High load average with high runnable tasks → CPU is saturated.
- High CPU usage with low load average → tasks may be I/O bound or waiting.

10. PRODUCTION QUESTION

CPU busy, CPU saturated, or workload blocked?

- Check run queue and load average.
- Look at CPU time breakdown (user, system, iowait, steal).
- Identify if the bottleneck is CPU, memory, storage, network or application.
- Use per-core metrics and process-level metrics to find the real cause.



SENIOR ENGINEER INSIGHT

High CPU usage does not automatically identify the root cause. Always check the run queue, load average, CPU time breakdown, blocked tasks, and I/O metrics to understand whether the CPU is truly saturated or the workload is waiting on something else.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



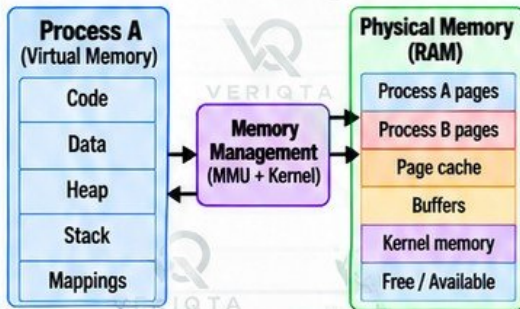
@veriqta

Linux Memory Anatomy

Understanding how Linux manages memory, what is used for, and how to interpret memory metrics

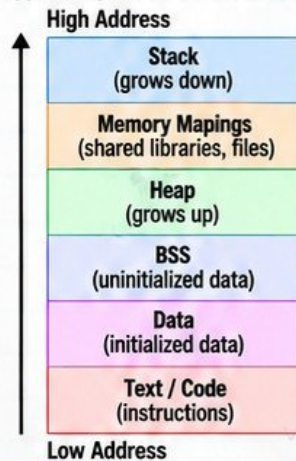
1. VIRTUAL VS PHYSICAL MEMORY

- Virtual memory gives each process its own isolated address space.
- Physical memory (RAM) is the actual hardware memory.
- The kernel maps virtual addresses to physical memory using page tables.
- This allows process isolation, memory protection and efficient usage.



2. PROCESS ADDRESS SPACES

- Each process has a separate virtual address space.
- Typical layout on 64-bit Linux:



3. ANONYMOUS MEMORY

- Anonymous memory is memory not backed by a file.
- Used for heap, stack and dynamically allocated memory (e.g. malloc).
- It can be swapped out to disk if needed.
- Shown as "AnonPages" in /proc/meminfo.

/proc/meminfo (anonymous memory)

```
MemTotal:      16384256 kB
MemFree:       124564 kB
MemAvailable:  4823040 kB
Buffers:       245760 kB
Cached:        3954688 kB
SwapCached:    1024 kB
Active(anon):  2211840 kB
Inactive(anon): 1056768 kB
AnonPages:     3268608 kB
...
```

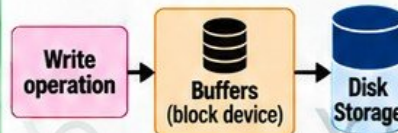
4. PAGE CACHE

- Page cache stores recently read file data in memory to speed up disk access.
- It is used automatically by the kernel and can be reclaimed when needed.
- Cached data can be dropped if applications need more memory.



5. BUFFERS

- Buffers are used for block device I/O (e.g. disk metadata).
- They store data before it is written to disk.
- Different from page cache, but also reclaimable.



6. FREE VS AVAILABLE MEMORY

- Free memory: RAM that is not used for anything.
- Available memory: memory that can be used for new applications without causing swapping. Includes reclaimable page cache and buffers.

Metric	Meaning
MemFree	Truly unused memory
MemAvailable	Memory available for new apps without swapping
Cached + Buffers	Reclaimable memory
Used	Memory used by processes, kernel, page cache and buffers

7. SWAP

- Swap is disk space used when RAM is not enough.
- Helps prevent out-of-memory (OOM) kills.
- Excessive swapping can severely impact performance.

```
$ swapon --show
```

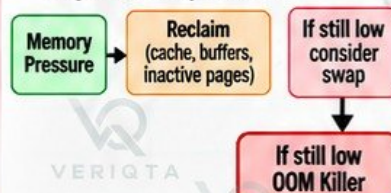
```
NAME      TYPE      SIZE  USED  PRIO
/dev/sda2 partition 4G    512M  -2
```

```
$ free -h
```

```
Mem:      total    used    free   available
Swap:    4.0G    512M    3.5G         -
```

8. MEMORY RECLAIM

- When memory is needed, the kernel reclaims memory from:
 - Page cache
 - Buffers
 - Inactive pages
 - Swappable anonymous memory
- If memory cannot be reclaimed and no swap is available, the OOM killer may terminate processes.



9. USEFUL COMMANDS

```
$ free -h
```

```
total    used    free   available
Mem:      15G    11G    124M    4.6G
Swap:     4.0G    512M    3.5G         -
```

```
$ cat /proc/meminfo | head -n 10
```

```
MemTotal:      16384256 kB
MemFree:       124564 kB
MemAvailable:  4823040 kB
Buffers:       245760 kB
Cached:        3954688 kB
SwapCached:    1024 kB
...
```

```
$ vmstat 1 5
```

```
procs  memory    swap    io      system    cpu
r  b  swpd free buff cache si so bi bo in cs us sy id wa st
1  0  0 134441 245760 3954688 0 0 12 34 56 78 5 3 88 4 0
0  0  0 12810 146000 3954688 0 0 10 28 52 71 4 2 91 3 0
```



10. SENIOR ENGINEER INSIGHT

Low "free" memory alone is not evidence of memory pressure.

- Linux will use available memory (page cache and buffers) to improve performance.
- Focus on MemAvailable, swap usage, reclaim behavior, application behavior and overall system performance.
- Investigate trends, not single numbers.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



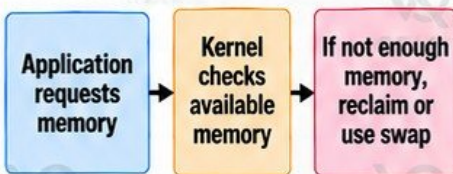
@veriqta

Memory Pressure and the OOM Killer

Understanding what happens when Linux runs out of memory and how the OOM killer works

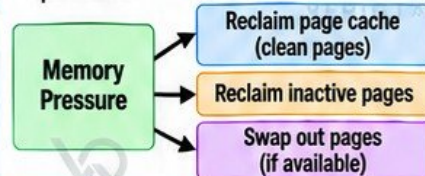
1. ALLOCATION PRESSURE

- Allocation pressure happens when processes request more memory than is available.
- The kernel tries to satisfy memory requests from free memory, page cache or by reclaiming memory.
- If memory cannot be reclaimed and no swap is available, the system reaches a critical state.



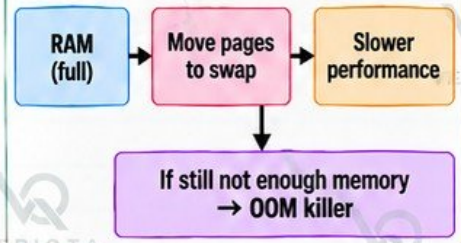
2. RECLAIM BEHAVIOR

- The kernel reclaims memory in this order:
 - Drop clean page cache
 - Reclaim unused buffers
 - Reclaim inactive pages
 - Swap out pages if swap is available.
- Active pages and locked memory cannot be easily reclaimed.
- Reclaim can cause I/O and increase latency if the system is under heavy pressure.



3. SWAPPING

- When RAM is insufficient, the kernel moves less recently used pages to swap.
- Swapping frees memory but is much slower than RAM access.
- Excessive swapping can cause severe performance degradation (thrashing).
- If both RAM and swap are insufficient, the OOM killer is triggered.



4. MAJOR PAGE FAULTS

- A major page fault occurs when a requested page is not in RAM and must be loaded from disk (e.g. swap or file).
- High major page fault rates indicate memory pressure and excessive swapping.
- This can significantly increase application latency.

```

$ vmstat 1 5
procs -----memory----- --swap--
r  b   swpd   free   buff   cache   si   so
5  0  1048576 125632 34588  987652 230 410
3  0  1048576  98032 30212  876540 320 510
  
```

High si/so values indicate heavy swapping and major page faults.

5. OOM SELECTION

- When memory cannot be reclaimed, the kernel's Out-Of-Memory (OOM) killer selects a process to kill.
- Processes with higher memory usage, higher oom_score, and lower protection are more likely to be killed.
- Critical system processes are less likely to be killed.

Key factors in OOM selection:

- Memory usage
- oom_score value
- oom_score_adj value
- Process importance
- Kernel protection rules

6. oom_score AND oom_score_adj

- oom_score shows how likely a process is to be killed (0 -1000).
- oom_score_adj allows you to adjust the score (-1000 to 1000).
- Lower values make the process less likely to be killed.
- Set critical processes to a lower score.

```

$ cat /proc/1234/oom_score
666
$ cat /proc/1234/oom_score_adj
0
  
```

oom_score_adj Value	Effect
-1000	Never kill (protected)
0	Default
1000	More likely to be killed

7. CGROUP MEMORY LIMITS

- cgroups allow you to set memory limits for processes, containers and services.
- If a process or container exceeds its memory limit, it can be killed by the OOM killer within that cgroup.
- Common in containers and orchestration platforms (Docker, Kubernetes).

```

$ cat /sys/fs/cgroup/memory_max
1073741824 # 1GB limit
$ cat /sys/fs/cgroup/memory.current
834338816 # current usage
  
```

8. CONTAINER OOM BEHAVIOR

- Containers share the host kernel, so the kernel OOM killer still applies.
- If a container exceeds its memory limit, the container process is killed.
- Depending on the configuration, the host or the container may be affected.
- In Kubernetes, this is seen as OOMKilled in the pod status.

```

$ kubectl get pods
NAME      STATUS    RESTARTS
web-7d9f5c OOMKilled 3
  
```

9. EVIDENCE FROM KERNEL LOGS

- When the OOM killer is triggered, the event is recorded in kernel logs.
- Logs show which process was killed, how much memory was used, and the system state at the time.

```

$ dmesg | grep -i -A5 "oom"
[ 12345.678] Out of memory: Killed process 4321 (python) total-vms:2097152kB, anon-rss:1048576kB, file-rss:0kB, shmem-rss:0kB, UID:1000 pgtables:2048kB oom_score_adj:0
[ 12345.679] oom_reaper: reaped process 4321 (python), now anon-rss:0kB
  
```

10. FAILURE FLOW



SENIOR ENGINEER INSIGHT

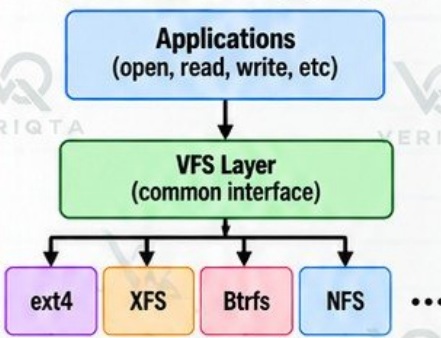
- Low "free" memory alone is not evidence of memory pressure.
- Always check available memory, swap usage, page fault rates and overall system behavior.
- Look at what is using memory and why.
- Understand the failure flow before taking action.

Filesystem Anatomy

Understanding how Linux stores, organizes and accesses files

1. VFS LAYER

- The Virtual Filesystem Switch (VFS) is an abstraction layer in the kernel.
- It provides a common interface for different filesystems (ext4, XFS, NFS, etc).
- Applications use the same system calls regardless of the underlying filesystem.



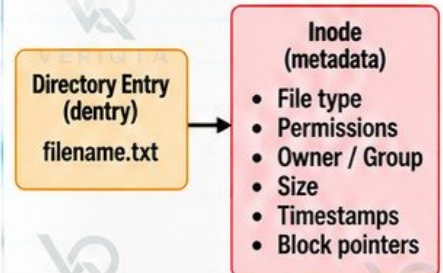
2. FILESYSTEMS AND MOUNT POINTS

- A filesystem organizes data on storage devices.
- It is made available at a mount point in the directory tree.
- Multiple filesystems can be mounted on different directories.

Filesystem	Common Use Case	Mount Point Example
ext4	General purpose (Linux default)	/, /home
XFS	High performance, large files	/data
Btrfs	Modern features (snapshots, RAID)	/var/lib
NFS	Network filesystem	/mnt/nfs
tmpfs	In-memory filesystem	/dev/shm

3. INODES AND DENTRIES

- Inodes store metadata about files (permissions, ownership, size, timestamps, block locations).
- Dentries (directory entries) map file names to inodes.
- A file name is just a reference to an inode.



4. FILE METADATA

- Files have metadata stored in the inode.
- Includes ownership, permissions, size, timestamps and extended attributes.

```

$ stat filename.txt
File: filename.txt
Size: 1024    Blocks: 8    IO Block: 4096
Device: 801h/2049d Inode: 262145 Links: 1
Access: 2026-10-07 10:00:00 +0000
Modify: 2026-10-07 09:50:00 +0000
Change: 2026-10-07 09:50:00 +0000
Birth: 2026-10-07 09:50:00 +0000
  
```

5. FILE DESCRIPTORS

- A file descriptor is a number used by a process to refer to an open file.
- Each process has its own file descriptor table.
- Common descriptors:

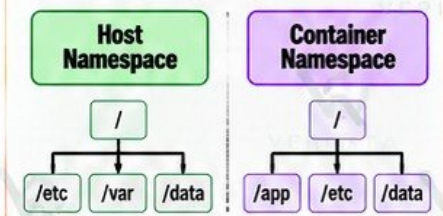
- 0 = stdin
- 1 = stdout
- 2 = stderr

File Descriptor Table	
0	stdin
1	stdout
2	stderr
3	/var/log/app.log
4	/etc/config.yaml
...	

Process

6. MOUNT NAMESPACES

- Mount namespaces provide isolated views of the filesystem.
- Each namespace can have different mount points.
- Containers use mount namespaces to isolate their filesystem from the host.



7. EXT4 AND XFS CONTEXT

ext4

- Widely used Linux filesystem.
- Journaling for reliability.
- Good general purpose performance.
- Common on most distributions.

XFS

- Designed for high performance and large files.
- Excellent parallel I/O.
- Common in enterprise environments.
- Often used for /data, databases and high-throughput workloads.

8. USEFUL COMMANDS

```

$ df -h # Show filesystem disk usage
$ du -sh /path # Show directory size
$ findmnt # Show mounted filesystems
$ mount # Show all mounts
$ stat file # Show file metadata
$ ls -li # Show inode number
$ df -li # Show inode usage
$ tune2fs -l /dev/sdX # ext4 info
$ xfs_info /dev/sdX # XFS info
  
```

9. COMMAND OUTPUT EXAMPLE

```

$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1       100G   40G   60G   40% /
/dev/sda2       500G  200G  300G   40% /data
tmpfs           4.0G    0    4.0G    0% /dev/shm
  
```

```

$ findmnt
TARGET SOURCE FSTYPE OPTIONS
/ /dev/sda1 ext4 rw,relatime
/data /dev/sdb1 xfs rw,relatime
/dev/shm tmpfs tmpfs rw,nosuid
  
```

10. PRODUCTION FAILURE



Filesystem reports space available but writes still fail.

- Common causes:
 - Inode exhaustion (no free inodes).
 - Disk quota limits.
 - Read-only filesystem (e.g. due to filesystem errors).
 - Underlying storage or network filesystem issues.
 - Container mount or overlay filesystem problems.

Example Symptoms

- df shows available space.
- Write operations fail with "No space left on device".
- Application logs show I/O errors.
- Filesystem may be read-only.
- Inodes may be 100% used.



SENIOR ENGINEER INSIGHT

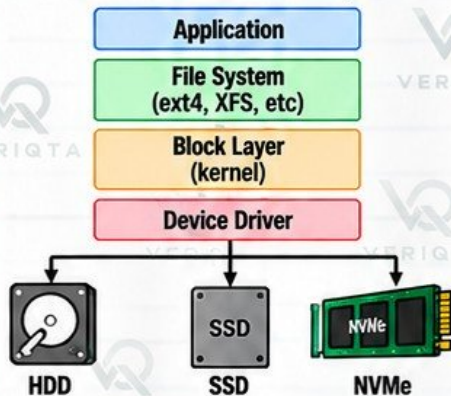
- A filesystem can report free space but still fail writes due to inode exhaustion, quotas, read-only state or underlying storage issues. Always check both disk space and inode usage, and verify the actual mount and filesystem state.

Disk I/O and Storage Saturation

Understanding how Linux handles disk I/O, what causes storage bottlenecks and how to identify them

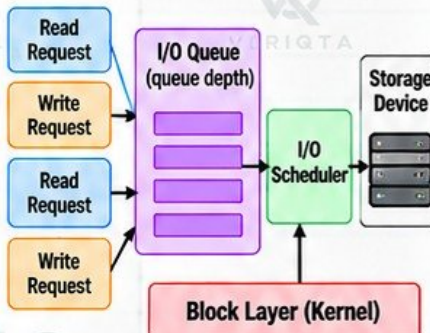
1. BLOCK DEVICES

- Disks and storage devices are accessed as block devices (e.g. /dev/sda).
- The kernel uses a block layer to manage I/O requests.
- Different device types: HDD, SSD, NVMe, network storage (iSCSI, NFS, etc).
- Block devices are organized into partitions and can be formatted with filesystems.



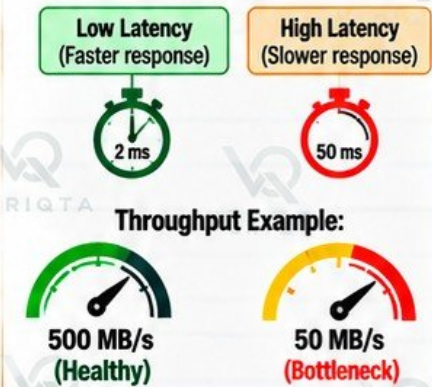
2. I/O REQUESTS AND QUEUES

- I/O requests are placed in queues before being processed by the storage device.
- The kernel uses an I/O scheduler to optimize request order (e.g. mq-deadline, kyber, none).
- Queue depth is the number of outstanding I/O requests.



3. LATENCY VS THROUGHPUT

- Latency:** time taken for a single I/O request to complete (milliseconds).
- Throughput:** total amount of data transferred per second (MB/s).
- Low latency + high throughput = healthy storage.
- High latency or low throughput indicates a bottleneck.



4. IOPS (INPUT/OUTPUT OPERATIONS PER SECOND)

- IOPS is the number of read/write operations completed per second.
- Crucial for databases, high-traffic applications and low-latency workloads.
- SSDs and NVMe devices provide much higher IOPS than traditional HDDs.

Storage Type	Typical IOPS	Typical Latency
HDD	100 – 500	5 – 15 ms
SATA SSD	10,000 – 50,000	0.1 – 1 ms
NVMe SSD	100,000+	0.02 – 0.2 ms

5. QUEUE DEPTH

- Queue depth is the number of I/O requests waiting to be processed.
- A high queue depth can indicate the storage device is saturated.
- Some storage devices handle high queue depths better than others (e.g. NVMe vs HDD).

Queue Depth Interpretation

- Low (0 – 1): idle or low load
- Moderate (2 – 10): normal
- High (> 10): storage may be saturated or slow

6. I/O WAIT

- I/O wait is the time the CPU spends waiting for disk I/O to complete.
- High iowait indicates storage bottlenecks.
- iowait does NOT mean high CPU usage, it means the CPU is idle waiting for I/O.

% iowait	Interpretation
< 5%	Normal
5 – 15%	Possible storage bottleneck
> 15%	Storage is likely saturated

7. READ/WRITE SATURATION

- Read saturation: storage cannot keep up with read requests.
- Write saturation: storage cannot keep up with write requests.
- Can be caused by high IOPS, large sequential writes, or slow underlying storage (e.g. network storage).

Signs of Saturation	Common Causes
- High latency - High queue depth - Low throughput - High iowait - Increased application latency	- Insufficient IOPS - Network storage latency - Disk contention (multiple VMs) - Slow underlying storage - Write-heavy workloads (logs, databases)

8. APPLICATION IMPACT

- Slow disk I/O affects:
 - Application response time
 - Database performance
 - Web server latency
 - Log writing and monitoring
 - Container and VM workloads

Common Symptoms

- Slow application responses
- Timeouts
- Increased error rates
- Backlog in queues
- Degraded user experience

9. USEFUL COMMANDS

```

$ iostat -xz 1 # disk I/O stats
$ lsblk -o NAME,SIZE,TYPE,MOUNTPOINT # list block devices
$ iotop # real-time I/O usage
$ pidstat -d 1 # per-process disk I/O
$ df -h # filesystem usage
$ du -sh /path # directory size
$ findmnt # show mount points
  
```

10. SENIOR REASONING



- High I/O wait or latency indicates a storage issue, not a CPU issue.
- Always check disk I/O metrics (iostat, iotop, pidstat -d) before investigating the application.
- Distinguish between storage latency and application latency.
- Correlate I/O metrics with application logs, database metrics, and network metrics.
- A slow application can be caused by storage, network or the application itself.
- Always gather evidence from multiple layers.

TROUBLESHOOTING FLOW

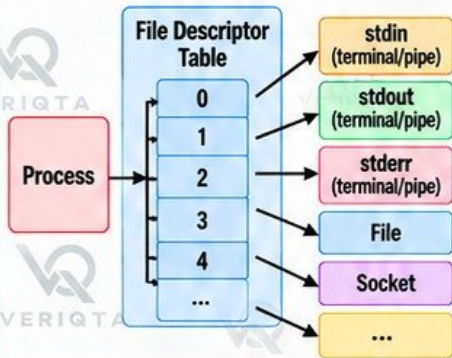


File Descriptors, Pipes and Open Files

Understanding how Linux represents open files, pipes, sockets and devices using file descriptors

1. FILE DESCRIPTOR TABLES

- Each process has a file descriptor (FD) table.
- A file descriptor is a number that refers to an open file, socket, pipe or device.
- The kernel maps the file descriptor to the actual underlying resource.
- Each process has its own FD table, isolated from other processes.



2. STDIN, STDOUT, STDERR

- Every process starts with three standard file descriptors:
 - 0 = **stdin** (standard input)
 - 1 = **stdout** (standard output)
 - 2 = **stderr** (standard error)
- They can point to a terminal, a file, a pipe or a socket.
- You can redirect them to files or pipes in shell and scripts.

FD	Name	Default Target	Common Use
0	stdin	Terminal	Read input
1	stdout	Terminal	Normal output
2	stderr	Terminal	Error output

Redirection Example

```
$ command > output.log 2> error.log
# stdout goes to output.log
# stderr goes to error.log
```

3. FILES, SOCKETS AND PIPES AS DESCRIPTORS

- Regular files, directories, sockets, pipes and devices are all accessed using file descriptors.
- The process does not care what type it is; it just uses the descriptor number.
- This unified model allows the same system calls (read, write, close, etc.) for different resource types.

	Regular File	e.g. /var/log/app.log
	Directory	e.g. /etc
	Device	e.g. /dev/null
	Pipe	e.g. between processes
	Socket	e.g. TCP/UDP connection
	Terminal / PTY	e.g. user session (SSH)

4. SOFT AND HARD LIMITS

- Each user and process has limits on the number of open file descriptors.
- Soft limit:** current limit for the process.
- Hard limit:** maximum limit that the soft limit can be raised to.
- If a process exceeds the limit, it will fail to open new files (e.g. "Too many open files").

Limit Type	Description
Soft Limit	Current limit for the process
Hard Limit	Maximum limit (requires root to increase)

5. ULIMIT

- ulimit** shows and sets resource limits for the current shell.
- Commonly used to view or set the open file descriptor limit.

```
$ ulimit -n # show soft limit
1024
$ ulimit -Hn # show hard limit
4096
$ ulimit -n 65536 # set new soft limit
```

Make it permanent (example for systemd service)

```
# /etc/systemd/system/myapp.service
[Service]
LimitNOFILE=65536
```

6. /PROC/<PID>/FD

- /proc/<pid>/fd** shows all open file descriptors for a process.
- Each descriptor is a symbolic link to the actual file, socket, pipe or device.

```
$ ls -l /proc/1234/fd
total 0
lrwx----- 1 user user 64 Oct 7 10:00 0 -> /dev/pts/1
l-wx----- 1 user user 64 Oct 7 10:00 1 -> /var/log/app.log
l-wx----- 1 user user 64 Oct 7 10:00 2 -> /var/log/app.err
lrwx----- 1 user user 64 Oct 7 10:00 3 -> socket:[12345]
l-wx----- 1 user user 64 Oct 7 10:00 4 -> pipe:[67890]
```

This helps you see exactly what resources a process has open.

7. DESCRIPTOR LEAKS

- A descriptor leak happens when a process opens files, sockets or pipes but does not close them.
- Over time, the number of open descriptors keeps increasing.
- Eventually the process hits the limit and cannot open new files or accept new connections.
- Common causes: bugs, missing `close()`, poor error handling, or long-running processes.

8. LSOF

- lsof** lists all open files for a process or system.
- Useful to find which files, sockets or devices are open and consuming descriptors.

```
$ lsof -p 1234
COMMAND PID USER FD TYPE NAME
myapp 1234 user 0 CHR /dev/pts/1
myapp 1234 user 1 REG /var/log/app.log
myapp 1234 user 2 REG /var/log/app.err
myapp 1234 user 3 IPv4 192.168.1.10:443
myapp 1234 user 4 PIPE pipe:[67890]
```

9. USEFUL COMMANDS

```
$ ulimit -n # show soft limit
$ ulimit -Hn # show hard limit
$ ulimit -n 65536 # set soft limit
$ ls -l /proc/<pid>/fd # list open fds
$ lsof -p <pid> # show open files
$ lsof -i # show open sockets
$ lsof | grep deleted # find deleted files
$ watch -n 1 "lsof -p <pid> | wc -l"
# monitor number of open fds
```

10. PRODUCTION FAILURE



Service is running but cannot accept new connections because file descriptors are exhausted.

- Symptoms: connection errors, "Too many open files", failed requests.
- Cause: descriptor leak or insufficient file descriptor limit.
- Impact: new connections and file operations fail, existing connections may still work.
- Fix: identify the leaking process, increase limits if needed, and fix the root cause.



SENIOR ENGINEER INSIGHT

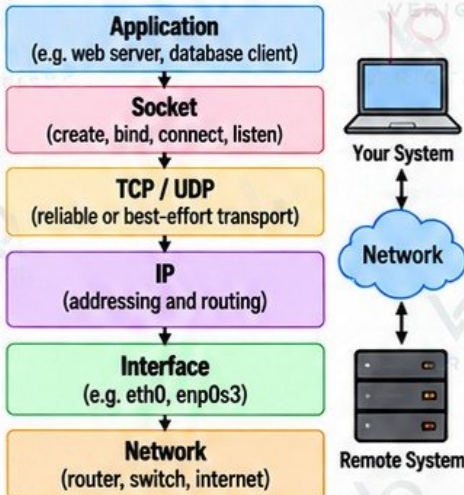
- High number of open file descriptors does not always mean a leak, but a constantly increasing number over time is a strong indicator.
- Check both per-process and system-wide usage.
- Distinguish descriptor exhaustion from other issues such as CPU, memory or network latency.
- Always correlate with application logs and metrics to understand the real impact.

Linux Networking Anatomy

Understanding how data travels from an application to a remote system in Linux

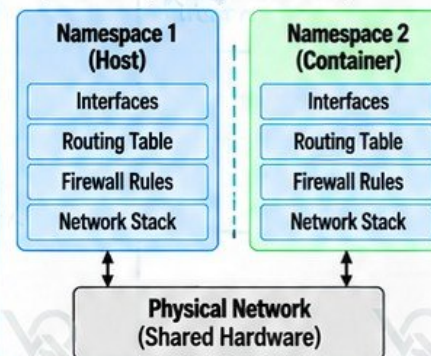
1. END-TO-END NETWORK FLOW

- Data flows through multiple layers from your application to a remote system.
- Each layer has a specific role.



2. NETWORK NAMESPACES

- Network namespaces provide isolated network stacks.
- Each namespace has its own:
 - Interfaces
 - Routing table
 - Firewall rules
 - Network configuration
- Used by containers and virtualization platforms (Docker, Kubernetes).



3. INTERFACES

- Network interfaces are the entry and exit points for network traffic.
- Each interface has an IP address, MAC address and status.
- Common interface names: eth0, enp0s3, ens33, wlan0, etc.

```

$ ip addr show
2: enp0s3: <BROADCAST,MULTICAST,UP>
   inet 192.168.1.10/24
   inet6 fe80::a00:27ff:fe4e:66a1/64
   link/ether 08:00:27:4e:66:a1
    
```

Item	Description
Interface Name	Logical name (e.g. eth0)
IP Address	Used for communication
MAC Address	Hardware address
State	UP or DOWN
MTU	Maximum packet size

4. ROUTING TABLE

- The routing table determines where packets are sent based on the destination IP address.
- It contains routes to local networks, remote networks and the default gateway.

```

$ ip route show
default via 192.168.1.1 dev enp0s3
192.168.1.0/24 dev enp0s3 proto kernel
10.0.0.0/16 via 192.168.1.254 dev enp0s3
    
```

Route Type	Description
default	Used for all unknown destinations
local	Routes to local networks
static	Manually configured routes
dynamic	Learned from routing protocols

5. ARP AND NEIGHBOR DISCOVERY

- ARP (Address Resolution Protocol) resolves an IP address to a MAC address on local networks (IPv4).
- For IPv6, neighbor discovery (NDP) performs a similar function.
- This is required to send packets to the next hop on the local network.



```

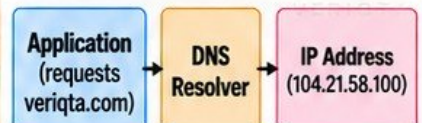
$ ip neigh show
IP address    dev    lladdr    state
192.168.1.1   enp0s3  08:00:27:4e:66:a1  REACHABLE
192.168.1.20  enp0s3  08:00:27:9a:12:3b  STALE
    
```

6. DNS RELATIONSHIP

- DNS translates human readable domain names to IP addresses.
- Applications use DNS to find the IP address of the remote system before establishing a connection.

```

$ dig veriqa.com +short
104.21.58.100
172.67.162.200
    
```



7. LISTENING SOCKETS

- Services listen on sockets to accept incoming connections.
- You can view listening sockets and the process using them.

```

$ ss -tulnp
Netid State Local Address:Port Process
tcp    LISTEN 0.0.0.0:22      1234/sshd
tcp    LISTEN 0.0.0.0:80      2345/nginx
udp    UNCONN 0.0.0.0:53      5678/dnsmasq
    
```

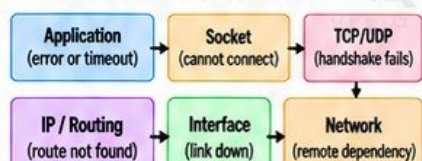
8. USEFUL COMMANDS

```

$ ip addr          # show interfaces
$ ip route         # show routing table
$ ip neigh         # show ARP/neighbor table
$ ss -tulnp        # show listening sockets
$ nslookup veriqa.com # DNS lookup
$ dig veriqa.com   # DNS lookup (detailed)
$ ping <ip>        # test connectivity
$ traceroute <ip>   # trace network path
$ curl -v <url>     # test HTTP/S connection
    
```

9. FAILURE-DOMAIN REASONING

- When a connection fails, check each layer from application to remote dependency.
- Identify which layer is failing to narrow down the root cause.



10. SENIOR ENGINEER INSIGHT



- Always follow the path from application to remote dependency.
- A network issue can be at the application, socket, DNS, routing, interface or remote system.
- Use the right tools at each layer to collect evidence.
- Check both local and remote sides.
- Do not assume the network is the problem without evidence.



@veriqa



@veriqa



@veriqa



@veriqa



@veriqa



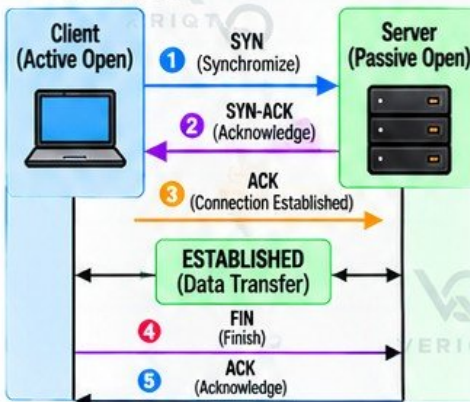
@veriqa

TCP Connections and Socket States

Understanding the TCP connection lifecycle, socket states and common production issues

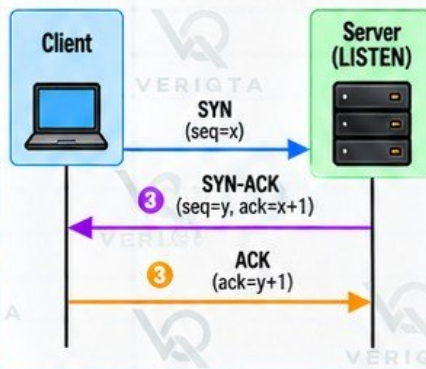
1. TCP CONNECTION LIFECYCLE

- TCP creates a reliable, ordered and bi-directional connection between a client and a server.
- It uses a sequence of states from connection establishment to teardown.
- Each state represents a specific step in the lifecycle.



2. TCP HANDSHAKE (SYN) BEHAVIOR

- The client sends a SYN to start a connection.
- The server responds with SYN-ACK if it is listening.
- The client sends an ACK to complete the handshake.
- If the server is not listening, a RST is returned.

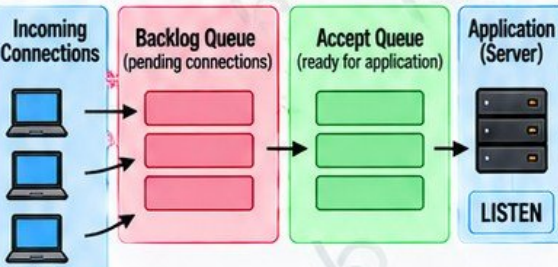


3. COMMON TCP SOCKET STATES

State	Description
LISTEN	Waiting for incoming connections.
SYN_SENT	SYN sent, waiting for SYN-ACK.
SYN_RECV	SYN received, waiting for ACK.
ESTABLISHED	Connection is established and data can be transferred.
FIN_WAIT_1	FIN sent, waiting for ACK.
FIN_WAIT_2	ACK received, waiting for FIN from the other side.
CLOSE_WAIT	Received FIN, waiting for application to close.
LAST_ACK	FIN sent, waiting for final ACK.
TIME_WAIT	Connection closed, waiting to ensure delayed packets are handled.

4. LISTEN STATE AND CONNECTION BACKLOG

- A socket in LISTEN state accepts incoming connections.
- The **backlog** is a queue for pending connections.
- If the backlog is full, new connection attempts may be dropped or refused.



5. TIME_WAIT AND EPHEMERAL PORTS

- TIME_WAIT helps ensure delayed packets do not affect new connections.
- It lasts for a period (typically $2 \times \text{MSL}$, often around 60 seconds).
- Clients use ephemeral ports for outgoing connections.
- If many connections are created and closed quickly, ephemeral ports can be exhausted.

Default Ephemeral Port Range (Linux)

32768 – 60999

6. CLOSE_WAIT STATE

- CLOSE_WAIT means the remote side has closed the connection.
- The local application has not closed its side yet.
- A large number of CLOSE_WAIT connections usually indicates an application issue (not closing sockets properly).



The application must close the socket to complete the connection teardown.

7. CONNECTION EXHAUSTION

- Too many open connections can exhaust system resources (file descriptors, memory, ports).
- This can prevent new connections from being accepted.
- Common signs: connection timeouts, refused connections, high CLOSE_WAIT or TIME_WAIT.

Common Causes

- Application not closing connections
- High traffic or load spikes
- Connection leaks
- Insufficient file descriptor limits
- Exhausted ephemeral ports

8. USEFUL COMMANDS

```

$ ss -s # summary of socket states
Total: 2345
TCP: 2001 (estab 1500, closed 300, orphaned 0, timewait 180)
UDP: 344
RAW: 0
FRAG: 0

$ ss -tan # list all TCP connections (numeric, all, no resolve)
State Recv-Q Send-Q Local Address:Port Peer Address:Port
LISTEN 0 128 0.0.0.0:80 0.0.0.0:*
ESTAB 0 0 10.0.0.5:54321 93.184.216.34:443
CLOSE-WAIT 0 0 10.0.0.5:5000 198.51.100.10:443
TIME-WAIT 0 0 10.0.0.5:60000 203.0.113.20:443
  
```

9. INTERPRETING OUTPUT

High number of specific states indicates a potential issue.

Observation	Likely Cause
Many SYN_RECV	High connection attempts or SYN flood.
Many CLOSE_WAIT	Application not closing connections properly.
Many TIME_WAIT	Normal for high connection churn. Can lead to ephemeral port exhaustion.
Cannot connect (new connections fail)	Backlog full, file descriptor limit reached, or ephemeral ports exhausted.

10. PRODUCTION FAILURE



Thousands of CLOSE_WAIT connections.

- This means the remote side has closed the connection, but the application is not closing its side.
- It usually indicates a bug or issue in the application (e.g. not calling close()).
- Over time, this can exhaust file descriptors and prevent new connections.

What to do:

- Identify the process with the CLOSE_WAIT connections (ss, lsof).
- Investigate the application code and logs.
- Check for resource leaks (file descriptors, memory).
- Fix the application to close connections properly.
- Increase limits temporarily if needed, but fix the root cause.



Senior Engineer's Insight

- A large number of CLOSE_WAIT connections is almost always an application issue, not a network issue. Focus on the application and its connection handling logic.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



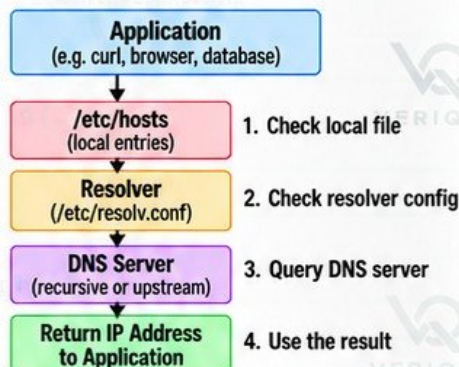
@veriqta

DNS and Name Resolution

Understanding how Linux resolves domain names, where it looks and how to troubleshoot issues

1. APPLICATION LOOKUP PATH

- When an application needs to resolve a domain name, it follows a specific lookup path.
- The system checks local configuration first, then uses configured DNS servers if needed.
- The result (IP address) is returned to the application, which then establishes a network connection.



2. /ETC/HOSTS

- Local static mapping of hostnames to IP addresses.
- Checked before querying DNS servers.
- Useful for testing, overrides and internal mappings.

```
$ cat /etc/hosts
127.0.0.1    localhost
::1         localhost ip6-localhost
192.168.1.10 db.example.com
10.0.0.5     api.internal.local
```

Entry Type	Description
IP address	The IP to return
Hostname	The domain name to match
Comments	Lines starting with # are ignored

3. /ETC/RESOLV.CONF

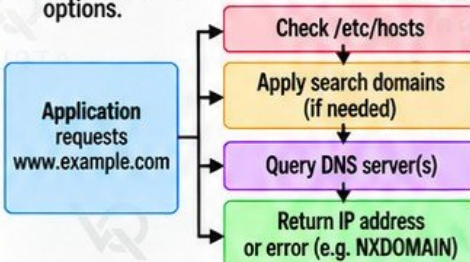
- Defines which DNS servers to use and search domains.
- Created and managed by systemd-resolved, NetworkManager or your network settings.
- Contains nameserver, search and options.

```
$ cat /etc/resolv.conf
nameserver 8.8.8.8
nameserver 1.1.1.1
search example.com internal.local
options timeout:2 attempts:3
```

Directive	Description
nameserver	DNS server IP address
search	Domains to append to short names
options	Resolver options (timeout, attempts, rotate, etc.)

4. RESOLVER BEHAVIOR

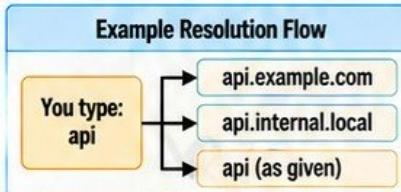
- The resolver checks /etc/hosts first.
- If not found, it queries the DNS servers listed in /etc/resolv.conf.
- It follows search domains for short names.
- It retries on failure based on configured options.



5. SEARCH DOMAINS

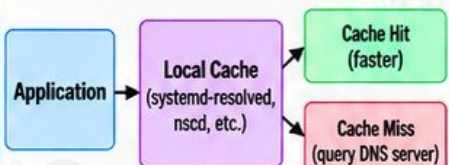
- If you use a short name (e.g. api), the resolver appends search domains.
- It tries each domain in order until it finds a match.

```
$ cat /etc/resolv.conf
search example.com internal.local
```



6. CACHING

- DNS responses are cached to improve performance.
- Caching can happen at different layers (systemd-resolved, nscd, local DNS, or application-level).
- Cached entries have a TTL (time to live).



7. DNS LATENCY

- DNS latency is the time taken to resolve a domain name to an IP address.
- High DNS latency can slow down application response times.
- It can be caused by slow DNS servers, network issues or misconfiguration.

Typical Latency	Description
< 20 ms	Normal (healthy)
20 – 100 ms	Slightly high (monitor)
> 100 ms	High (investigate)

8. NXDOMAIN

- NXDOMAIN means the domain name does not exist.
- It can be caused by a typo, incorrect domain or missing DNS record.

```
$ dig veriqttaa.com +short
;;->HEADER<- opcode: QUERY,
status: NXDOMAIN,
id: 12345
;; QUESTION SECTION:
;veriqttaa.com.      IN  A
```

9. TIMEOUTS AND RETRIES

- If a DNS server does not respond, the resolver waits for a timeout period.
- It then retries based on the configured attempts.
- This can lead to delays if DNS servers are slow or unreachable.

```
$ cat /etc/resolv.conf
options timeout:2 attempts:3
```

Option	Description
timeout:n	Time to wait for a response (seconds)
attempts:n	Number of retries before giving up

10. USEFUL COMMANDS

```
# DNS lookup
$ dig veriqtta.com
$ dig veriqtta.com +short

# Check name resolution
$ getent hosts veriqtta.com

# Check resolver status
$ resolvectl status

# Test DNS server directly
$ dig @8.8.8.8 veriqtta.com
$ dig veriqtta.com +trace
```

11. INCIDENT LESSON

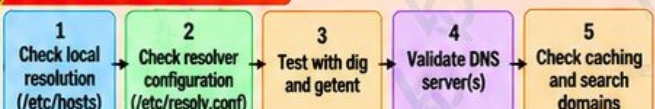


"Network failure" can actually be a name-resolution failure.

- Applications may show connection errors when DNS fails.
- Always check name resolution (dig, getent, resolvectl) before assuming a network or application issue.
- DNS issues can cause timeouts, slow responses or complete outages even when the network and target service are healthy.



TROUBLESHOOTING FLOW



Users, Permissions and Privilege

Understanding how Linux manages users, groups, permissions and access control

1. UID AND GID

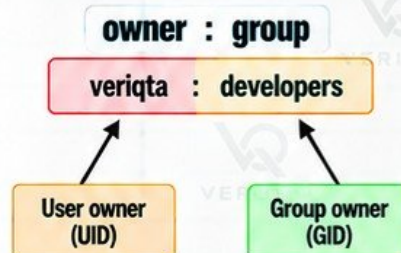
- Each user has a User ID (UID).
- Each group has a Group ID (GID).
- UID and GID are numeric identifiers used by the kernel.
- UID 0 is the root user.
- Regular users typically start from UID 1000.

Account	UID	GID	Description
root	0	0	Superuser
system	1 - 999	1 - 999	System accounts (services)
user	1000+	1000+	Regular users

```
$ id veriqa
uid=1001(veriqa) gid=1001(veriqa)
groups=1001(veriqa),27(sudo)
```

2. OWNERSHIP

- Every file and directory has an owner (user) and a group.
- Ownership controls who can access and modify the file.



```
$ ls -l /var/www
drwxr-xr-x 5 veriqa developers 4096 Oct 7 18:00 /var/www
```

3. MODE BITS (PERMISSIONS)

- Linux uses mode bits to control access:
- Read (r) = 4
- Write (w) = 2
- Execute (x) = 1
- Permissions are set for user (u), group (g) and others (o).

r	w	x	r	w	x	r	w	x
User (7)			Group (5)			Others (5)		

Example: 755 = rwxr-xr-x

Symbolic	Numeric	Meaning
rwx	7	Read, write, execute
rw-	6	Read, write
r-x	5	Read, execute
r--	4	Read
---	0	No access

4. EFFECTIVE IDENTITY

- A process has a real user ID (who started it) and an effective user ID (what it runs as).
- The effective identity determines the permissions used for file and system access.

Type	Description
Real UID (RUID)	The actual user who started the process.
Effective UID (EUID)	The user identity used for permissions.
Effective GID (EGID)	The group identity used for permissions.

```
$ id
uid=1001(veriqa) gid=1001(veriqa)
euid=0(root) egid=0(root) # running with sudo
```

5. SUDO

- sudo allows authorized users to run commands as another user (usually root).
- Provides controlled, audited access.
- Configured in /etc/sudoers.

```
$ sudo systemctl restart nginx
```

Common sudo use cases

- Run a single command as root
- Restart services
- Edit system files
- Switch to a root shell (sudo -i)

6. LINUX CAPABILITIES

- Linux capabilities allow specific administrative privileges without full root access.
- Useful for giving fine-grained permissions to applications.

```
$ getcap /usr/bin/ping
/usr/bin/ping = cap_net_raw+ep
```

Capability	Use Case
cap_net_raw	Required for ping and raw network access
cap_net_bind_service	Bind to ports < 1024
cap_sys_admin	Many administrative operations (mount, namespaces, etc.)

7. SUID AND SGID

- SUID allows a file to run with the owner's effective user ID.
- SGID allows a file to run with the group's effective group ID.
- Common for specific system binaries (e.g. passwd, ping).

Permission	Symbol	Example	Meaning
SUID	s	4755	Runs with owner's privileges
SGID	s	2755	Runs with group's privileges

```
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 68208 Sep 10 2023 /usr/bin/passwd
```

8. ACLs (ACCESS CONTROL LISTS)

- ACLs provide more granular permissions than traditional mode bits.
- Allow different permissions for multiple users and groups.

```
$ getfacl /var/www
# file: /var/www
# owner: veriqa
# group: developers
user::rwx
user:deploy:r-x
group::r-x
group:qa:r-x
mask::r-x
other::r-x
```

9. SERVICE IDENTITIES

- Services should run with dedicated, limited-privilege users.
- Avoid running application services as root.
- Use system users with minimal permissions.

Service	Typical User
nginx	nginx
mysql	mysql
postgres	postgres
redis	redis
docker	root (uses group docker)

10. LEAST PRIVILEGE

- Give only the permissions required to perform a task.
- Reduces the impact of security breaches.
- Use dedicated service accounts.
- Avoid unnecessary root access.
- Review permissions regularly.

- Create dedicated users
- Set minimal permissions
- Use sudo with restrictions
- Apply capabilities where possible
- Remove unnecessary privileges

11. SECURITY REMINDER



Root access should not be the default fix for permission problems.

- Understand the required permissions before changing ownership or using sudo.
- Use the least privilege necessary.
- Running as root increases the risk of mistakes and security incidents.
- Always investigate the real cause of a permission issue.



COMMON PERMISSION COMMANDS

```
$ ls -l
$ chmod 755 file
$ chown user:group file
$ getfacl file
$ setfacl -m u:user:rwx file
$ sudo -l
$ getcap file
```

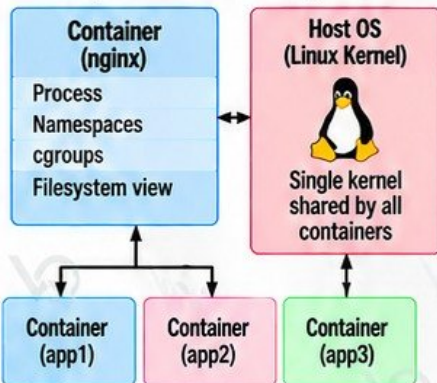
view permissions
change mode bits
change ownership
view ACLs
set ACL for a user
show allowed sudo commands
view file capabilities

Namespaces, cgroups and Container Foundations

Understanding how Linux namespaces and cgroups make containers possible

1. CONTAINERS ARE LINUX PROCESSES

- Containers are regular Linux processes running on the host kernel.
- They are not miniature virtual machines.
- They share the host kernel but use namespaces for isolation and cgroups for resource control.
- Containers package the application, its dependencies and configuration.



2. LINUX NAMESPACES OVERVIEW

- Namespaces provide isolation by giving each container its own view of certain system resources.
- Each namespace type isolates a different part of the system.
- Multiple namespaces work together to create container isolation.

Namespace	Isolates	What it provides
PID	Process IDs	Own process tree (PID 1 inside container)
Network	Network stack	Own network interfaces, routing, firewall rules
Mount	Filesystem view	Own filesystem tree (mount points)
User	User and group IDs	Maps container users to host users
UTS	Hostname	Own hostname and domain name
IPC	Inter-process communication	Own shared memory, message queues, semaphores

3. PID NAMESPACES

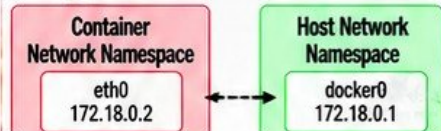
- Gives the container its own process ID space.
- The first process inside the container appears as PID 1.
- Processes outside the namespace are not visible.

```

$ docker run --rm -it nginx ps aux
PID USER %CPU %MEM COMMAND
1 root 0.0 0.1 nginx
7 nginx 0.0 0.0 nginx: worker
$ ps -ef | grep nginx # on host
12345 root ... nginx (container)
  
```

4. NETWORK NAMESPACES

- Provides an isolated network stack for the container.
- Each container has its own interfaces, IP addresses, routing table and firewall rules.



5. MOUNT NAMESPACES

- Gives the container its own filesystem view.
- Allows different mount points and filesystems from the host.
- Changes inside the container do not affect the host (unless explicitly shared).

```

$ docker run --rm --it nginx mount | head
overlay on / type overlay (rw,relatime,lowerdir
...
proc on /proc type proc (rw,nosuid,nodev,noexec)
...
  
```

6. USER NAMESPACES

- Isolates user and group IDs.
- Maps container root to a non-root user on the host.
- Improves security by reducing the impact if a container is compromised.

Example Mapping

Container root (UID 0)
↓ (mapped to)
Host UID 100000

This means root inside the container is not root on the host.

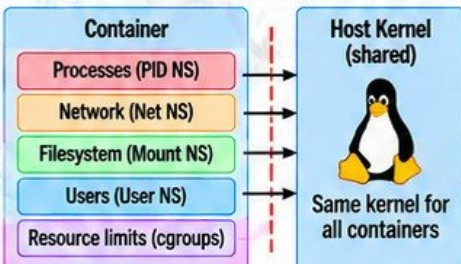
7. CGROUPS

- cgroups (control groups) limit, account and manage resource usage.
- Used to control CPU, memory, I/O and other resources for containers.

Resource	What cgroups control	Example
CPU	CPU shares and limits	--cpus=1
Memory	Memory usage and limits	--memory=512m
Block I/O	Disk I/O bandwidth	--blkio-weight=500
PIDs	Number of processes	--pids-limit=100
Devices	Access to devices	Allow or deny

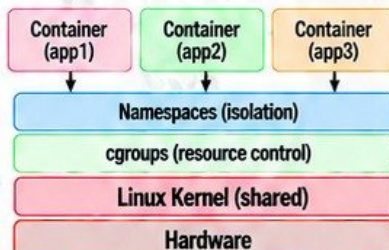
8. CONTAINER ISOLATION BOUNDARIES

- Namespaces isolate what the container can see.
- cgroups limit what the container can use.
- Containers are isolated but still share the host kernel.
- They are not a security boundary like a full VM.



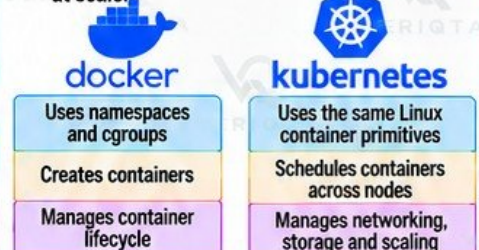
9. HOST KERNEL SHARING

- All containers share the same Linux kernel.
- They use namespaces and cgroups for isolation, not a separate kernel.
- Kernel vulnerabilities can affect containers.
- Keep the host kernel updated.



10. CONNECTION TO DOCKER AND KUBERNETES

- Docker uses Linux namespaces and cgroups to create and manage containers.
- Kubernetes builds on the same Linux features for container orchestration at scale.



11. TROUBLESHOOTING TIPS



- Check container namespaces using: `lsns`
- Inspect cgroup usage: `cat /sys/fs/cgroup/`
- Use `docker inspect <container>` to see namespace and cgroup settings.
- When debugging, check both inside the container and on the host.
- Many container issues are easier to understand from the host.

12. SENIOR ENGINEER INSIGHT



Container debugging often requires understanding the host beneath it.

- Containers share the host kernel, so host-level tools and logs are essential.
- Always investigate from both the container and the host.
- Understanding namespaces, cgroups and kernel behavior is critical for resolving complex container issues.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



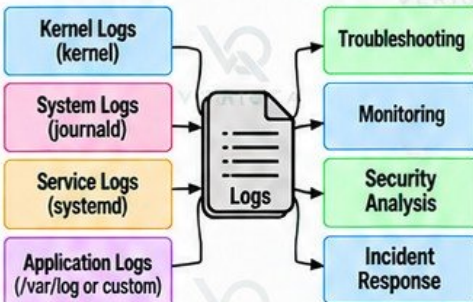
@veriqta

Linux Logging and Observability

Understanding logs, metrics and system signals to troubleshoot and operate Linux systems

1. LINUX LOGGING OVERVIEW

- Linux generates logs from the kernel, system services, applications and security components.
- Logs help you understand what happened, when it happened and why.
- They are essential for troubleshooting, monitoring and incident response.



2. KERNEL LOGS

- The kernel logs hardware, driver, boot and system-level events.
- Stored in a ring buffer and exposed through journald or dmesg.
- Important for understanding system failures, reboots and hardware issues.

```
$ dmesg | less # view kernel ring buffer
$ dmesg -T    # human readable timestamps
$ journalctl -k # kernel logs via journald
```

Common Kernel Log Topics	Examples
Hardware detection	CPU, memory, disk, PCI devices
Driver issues	failed to load, device errors
System events	reboot, shutdown, OOM, panics
Network events	interface up/down, link changes

3. JOURNALD

- systemd-journald collects and manages logs from the kernel, services and applications.
- Logs are structured with metadata (timestamp, service, PID, etc).
- Use journalctl to query and filter logs.

```
$ journalctl -xe # view recent logs
$ journalctl -u nginx # logs for a service
$ journalctl --since "1 hour ago"
$ journalctl -p err # only error logs
```

Useful journalctl Options	Description
-u <service>	Filter by service name
-k	Show kernel logs
--since "time"	Show logs from a time
-p <level>	Filter by priority (err, warn, info)
-f	Follow logs in real time

4. APPLICATION LOGS

- Applications write their own logs, usually to files in /var/log or a custom path.
- Log format and location depend on the application (e.g. Nginx, Apache, databases, web apps).
- These logs show application-level events, requests, errors and performance issues.

```
$ tail -f /var/log/nginx/access.log
$ tail -f /var/log/nginx/error.log
$ tail -f /var/log/httpd/error_log
$ tail -f /var/log/mysql/error.log
```

5. SERVICE LOGS

- System services managed by systemd write logs to journald.
- You can view service-specific logs using journalctl -u.
- Helps you troubleshoot service startup, crashes and runtime issues.

```
$ systemctl status nginx # service status
$ journalctl -u nginx # view service logs
$ journalctl -u ssh -f # follow logs
```

6. METRICS

- Metrics are numerical data collected over time (e.g. CPU, memory, disk, network, request rate).
- They help you understand system health, performance and trends.
- Common tools: top, vmstat, iostat, sar, prometheus node exporter, etc.



7. PROCESS SIGNALS

- Signals are used to notify processes about events (e.g. terminate, reload, stop).
- Common signals: SIGTERM, SIGKILL, SIGHUP, SIGUSR1, SIGUSR2.
- Useful for controlling and troubleshooting services.

Signal	Number	Use Case
SIGTERM	15	Graceful shutdown (default)
SIGKILL	9	Force kill (cannot be ignored)
SIGHUP	1	Reload configuration
SIGUSR1	10	Application specific action
SIGUSR2	12	Application specific action

```
$ kill -15 <pid> # send SIGTERM
$ kill -9 <pid> # send SIGKILL
```

8. RESOURCE TELEMETRY

- Resource telemetry shows how system resources are being used over time.
- These metrics help you understand system behavior and quickly identify issues.

Tool	Purpose	Example
top	Real-time process view	top
htop	Interactive process view	htop
vmstat	CPU, memory, I/O stats	vmstat 1
iostat	Disk I/O stats	iostat -x 1
sar	Historical performance	sar -u 1 5
ss	Network connection stats	ss -s

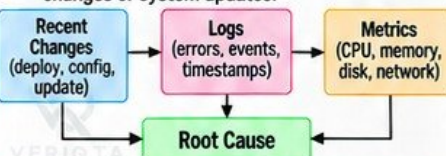
9. /VAR/LOG DIRECTORY

- Traditional log files are stored in /var/log.
- Contains logs for system, services and applications.
- Log rotation (logrotate) is used to manage file size and retention.

```
$ ls -l /var/log
auth.log # authentication logs
syslog # general system logs
kern.log # kernel logs
nginx/ # application logs
mysql/ # database logs
messages # system messages
secure # security related logs
```

10. CORRELATING LOGS WITH METRICS

- During troubleshooting, correlate logs with metrics and recent changes.
- Check logs at the same time as spikes in CPU, memory, disk I/O or network traffic.
- Also review recent deployments, configuration changes or system updates.



11. OBSERVABILITY MODEL

- A good observability model uses three key signals: latency, traffic, errors and saturation.
- These metrics help you understand system behavior and quickly identify issues.

Signal	What it tells you	Examples
Latency	How long requests take	Response time, p95/p99
Traffic	How much load the system is handling	Requests per second (RPS)
Errors	Whether requests are failing	Error rate, HTTP 5xx
Saturation	Whether the system is running out of resources	CPU, memory, disk I/O, network utilization

12. OPERATIONAL RISK

Having logs is not the same as having useful observability.

- Logs may be missing, incomplete or not retained long enough.
- No metrics means you cannot see the full picture of system health.
- Without proper observability, issues are harder to detect, troubleshoot and resolve.

Senior Engineer Insight

Container debugging often requires understanding the host beneath it. Logs, metrics and system behavior on the host can reveal issues that are not visible inside the container.

Linux Security Boundaries

Understanding how Linux isolates, restricts and protects systems, processes and resources

1. DAC PERMISSIONS

- Discretionary Access Control (DAC) uses user and group ownership and mode bits to control access to files and directories.
- It is the default permission model in Linux.
- Simple but essential for everyday access control.

	r	w	x
Owner (u)	r	w	x
Group (g)	r	w	x
Others (o)	r	w	x

Example: 755 = `rw-r-xr-x`

```
$ ls -l file.txt
-rwxr-xr-- 1 user group 1024 Oct 7 10:00 file.txt
```

2. LINUX CAPABILITIES

- Capabilities break down root privileges into smaller, fine-grained permissions.
- Allows specific privileges without full root access.
- Useful for running services with only the permissions they need.

Capability	Use Case
cap_net_bind_service	Bind to ports < 1024
cap_net_admin	Configure network settings
cap_sys_admin	Many administrative tasks (mount, namespaces, etc.)
cap_dac_override	Bypass file permissions

```
$ getcap /usr/bin/ping
/usr/bin/ping = cap_net_raw+ep
```

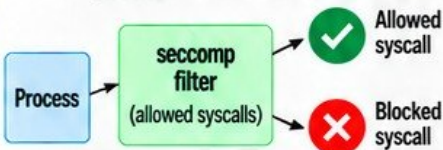
3. SELINUX / APPARMOR

- SELinux and AppArmor are mandatory access control (MAC) systems.
- They enforce security policies that limit what processes can do, even if running as root.
- Helps prevent unauthorized access and limits the impact of compromised services.

SELinux (RHEL, CentOS, Fedora)	AppArmor (Ubuntu, Debian)
- Mandatory access control - Policy based - Enforces process confinement - Modes: enforcing, permissive, disabled	- Mandatory access control - Profile based - Easier to configure - Controls file, network and capability access
\$ getenforce Enforcing	\$ sudo aa-status apparmor module is loaded

4. SECCOMP

- seccomp (secure computing mode) restricts the system calls a process can use.
- Helps reduce the attack surface of applications.
- Widely used in containers and sandboxes.



```
$ grep Seccomp /proc/<pid>/status
Seccomp: 2 (filter mode)
```

5. NAMESPACES

- Namespaces provide isolated views of system resources.
- Each container can have its own processes, network stack, filesystem, users, etc.
- Helps multiple containers run on the same host without interfering.

Namespace	Isolates
PID	Process IDs
Network	Network stack, interfaces, routing
Mount	Filesystem view (mount points)
User	User and group IDs
UTS	Hostname and domain name
IPC	Inter-process communication

6. CGROUPS AS RESOURCE BOUNDARIES

- Control groups (cgroups) limit, account and manage resource usage.
- Used to restrict CPU, memory, I/O and other resources for processes and containers.
- Helps prevent one workload from consuming all system resources.

Resource	Control with cgroups
CPU	Limit CPU shares and usage
Memory	Limit memory usage and set OOM behavior
Block I/O	Limit disk I/O bandwidth
PIDs	Limit number of processes
Devices	Allow or deny device access

```
$ cat /sys/fs/cgroup/memory/memory.limit_in_bytes
1073741824
$ cat /sys/fs/cgroup/cpu/cpu.shares
1024
```

7. FIREWALLING

- Controls inbound and outbound network traffic.
- Common tools: nftables (modern), iptables (legacy), firewalld (management layer).
- Helps allow only required traffic and block unauthorized access.

```
# Using nftables (modern)
$ sudo nft list ruleet

# Using iptables (legacy)
$ sudo iptables -L -n -v
```



8. AUDIT LOGS

- Audit logs record security-relevant events such as file access, privilege changes and system calls.
- Helps with incident investigation, compliance and detecting suspicious activity.

```
$ sudo auditctl -l # list audit rules
$ sudo ausearch -m USER_CMD # search commands
$ sudo aureport -summary # audit summary
```

Common Events	Audit Type
File access	-w /path -p rwx
Privilege changes	-a always,exit -F auid>=1000
Execution of commands	-a always,exit -F arch=b64 -S execve
Login/logout	-a always,exit -F type=USER_LOGIN

9. PRIVILEGE ESCALATION PATHS

- Attackers try to gain higher privileges through misconfigurations, kernel vulnerabilities or weak permissions.
- Common paths: SUID binaries, writable system files, misconfigured sudo, weak capabilities, container escapes.

Common Escalation Paths

- SUID/SGID binaries
- Excessive capabilities
- Misconfigured sudo rules
- Writable files or directories
- Kernel vulnerabilities
- Container breakout (namespaces, cgroups)



10. DEFENSE IN DEPTH

- Use multiple layers of security to reduce risk.
- Combine DAC, capabilities, MAC (SELinux/AppArmor), namespaces, cgroups, firewalling and audit logging.
- If one layer fails, other layers help limit the damage.



11. ARCHITECTURE TRADE-OFF

- Stronger isolation improves security but can increase operational complexity.
- More layers mean more configuration, monitoring and troubleshooting.
- The right balance depends on your environment, risk tolerance and team expertise.

Stronger Isolation
(More Secure)



Operational Complexity
(More Overhead)

12. SENIOR ENGINEER INSIGHT

Container debugging often requires understanding the host beneath it.

- Containers rely on the host kernel, namespaces and cgroups.
- Many security issues are visible only from the host (e.g. kernel logs, cgroups, namespaces, audit logs).
- Always investigate both the container and the host when troubleshooting security or isolation issues.

Production Linux Failure Investigation

Real Incident: API latency rises sharply while several services begin timing out.

INVESTIGATION PATH		WHAT TO CHECK (EVIDENCE)	KEY COMMANDS (EXAMPLES)	WHAT IT TELLS YOU
1	 SYMPTOM API latency rises, services timing out.	- Check monitoring dashboards - Confirm error rates and latency - Identify affected services	<pre>\$ curl -w "time:%{time_total}\n" https://api.example.com \$ kubectl get pods -A (or systemctl status)</pre>	Confirms the issue and shows the initial scope (e.g. high latency, 5xx errors, timeouts).
2	 DEFINE BLAST RADIUS Understand impact.	- Which services are affected? - Internal vs external? - Single host or multiple hosts?	<pre>\$ kubectl get pods -A \$ systemctl --failed \$ curl -I http://service:port</pre>	Shows how widespread the issue is and which services or users are affected.
3	 CHECK RECENT CHANGES Look for what changed.	- Deployments, config changes - Package updates, reboots - Infrastructure or network changes	<pre>\$ git log --oneline \$ kubectl rollout history deployment/<name> \$ yum history tail (or apt history) \$ journalctl -k --since "2 hours ago"</pre>	Identifies recent changes that could have introduced the issue.
4	 CPU AND RUN QUEUE Check CPU pressure.	- High CPU usage? - High run queue? - CPU throttling (containers)?	<pre>\$ top \$ uptime \$ sar -u 1 5 \$ cat /sys/fs/cgroup/cpu/cpu.stat</pre>	Shows if CPU is saturated or processes are waiting in the run queue.
5	 MEMORY PRESSURE Check memory usage.	- High memory usage? - Swapping? - OOM kills?	<pre>\$ free -h \$ vmstat 1 5 \$ sar -r 1 5 \$ dmesg grep -i oom</pre>	Confirms memory pressure, swapping or OOM events that can cause latency and timeouts.
6	 DISK LATENCY Check storage performance.	- High disk I/O or latency? - Full filesystem? - I/O wait high?	<pre>\$ iostat -x 1 5 \$ iotop -o \$ df -h \$ dmesg grep -i -E "sd nvme i/o error"</pre>	Identifies disk bottlenecks or storage errors causing slow responses.
7	 SOCKETS AND CONNECTION STATE Check network connections.	- Many connections in TIME_WAIT? - High ESTABLISHED or CLOSE_WAIT? - Connection exhaustion?	<pre>\$ ss -s \$ ss -tan head \$ ss -tan state time-wait wc -l \$ netstat -s grep -i listen</pre>	Shows connection state, backlog and potential connection leaks or exhaustion.
8	 DNS AND DEPENDENCIES Check name resolution and external services.	- DNS resolution slow or failing? - External dependencies timing out? - Network reachability issues?	<pre>\$ dig api.example.com \$ getent hosts api.example.com \$ ping -c 4 api.example.com \$ curl -v https://dependency.com</pre>	Confirms if DNS or external dependencies are causing timeouts and latency.
9	 KERNEL AND SERVICE LOGS Look for errors and clues.	- Kernel errors, OOM, I/O errors? - Service errors and timeouts? - Resource limits being hit?	<pre>\$ journalctl -xe \$ journalctl -u <service> --since "2 hours ago" \$ dmesg -T tail -n 50 \$ grep -i error /var/log/<service>.log</pre>	Reveals specific errors, warnings and events related to the failure.
10	 IDENTIFY ROOT CAUSE Confirm the real cause.	- Correlate findings from all areas - Validate theory with evidence - Rule out other possible causes	Use the relevant commands above to confirm the most likely cause.	Pinpoints the actual cause (e.g. CPU saturation, DB slow, DNS issue, bug, etc).
11	 MITIGATE Reduce impact.	- Restart or scale services - Rollback recent changes - Clear stuck connections, free resources	<pre>\$ systemctl restart <service> \$ kubectl rollout undo deployment/<name> \$ kill <pid> (if safe) \$ echo 3 > /proc/sys/vm/drop_caches</pre>	Restores service and reduces user impact.
12	 VERIFY RECOVERY Ensure the issue is resolved.	- Check latency, error rates - Confirm services are healthy - Monitor for stability	<pre>\$ curl -w "time:%{time_total}\n" https://api.example.com \$ systemctl status <service> \$ kubectl get pods -A \$ top, iostat, ss -s</pre>	Confirms the system has recovered and is stable.
13	 PREVENT RECURRENCE Fix the root issue for good.	- Implement permanent fix - Add monitoring and alerts - Improve capacity, configuration or code	<pre>\$ Update configs, apply patches \$ Set up alerts (Prometheus, etc) \$ Document the incident and actions</pre>	Reduces the chance of the same issue happening again.
14. INCIDENT EXAMPLE SUMMARY		15. KEY TAKEAWAYS		16. SENIOR ENGINEER PRINCIPLE
- API latency increased from 100ms to 8s. - Several services started timing out. - Investigation found high CPU usage due to a new background process after a recent deployment. - Killing the process and rolling back the change restored normal latency. Alerts and monitoring added to prevent recurrence.		- Always follow a structured investigation path. - Use real evidence, not assumptions. - Check the whole system: CPU, memory, disk, network, logs and dependencies. - Correlate multiple signals before deciding. - Document the root cause and fix. - Improve monitoring and alerting.		Do not ask, "Which command should I run?" Ask, "Which subsystem could produce this symptom, and what evidence would prove or eliminate that hypothesis?"



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Reading the Whole Linux System

From application to hardware with operational overlays

1. APPLICATION

- User requests, API calls, jobs, web services, databases, background workers.
- Generates processes, files, network connections and logs.
- Symptoms appear here first (e.g. high latency, errors, timeouts).

Examples



Application

(Web, API, Database, Workers, etc.)



Process and Threads

(PID, memory space, open files, threads)

2. PROCESS AND THREADS

- Applications run as processes with one or more threads.
- Each process has its own memory space and resources.
- Threads share the same process memory.

Key Concepts



Process (PID)



Threads (lighter weight)



Process resources

3. SYSTEM CALLS

- Programs use system calls to request services from the kernel.
- Examples: read, write, open, close, fork, exec, socket, etc.
- System calls are the interface between user space and kernel space.

Common System Calls

read()	write()	open()
close()	fork()	exec()
socket()	connect()	stat()



System Calls

(Interface to the kernel)



Kernel

(Process management, memory, I/O, networking, filesystem, security)

4. KERNEL

- Manages CPU, memory, I/O, filesystems, networking and devices.
- Enforces security, isolation and resource control.
- Handles system calls and schedules work on hardware.

Key Responsibilities

- | | |
|--------------------------|------------------------|
| • Process scheduling | • Filesystem layer |
| • Memory management | • Security enforcement |
| • I/O and device drivers | • Resource isolation |
| • Networking stack | • Interrupt handling |

5. CPU, MEMORY AND I/O

- Kernel schedules processes on CPU, manages memory and handles I/O requests.
- Bottlenecks here can cause latency, timeouts and failures.

Examples



CPU

(Scheduling, run queue, load)



Memory

(Allocation, page cache, swapping)



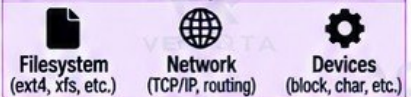
I/O

(Disk, network, devices)

6. FILESYSTEM, NETWORK AND DEVICES

- Filesystem: manages files, directories and storage.
- Network: handles connections, routing and communication.
- Devices: access to physical and virtual hardware.

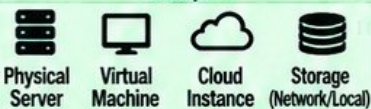
Examples



7. HARDWARE / VIRTUAL INFRASTRUCTURE

- Physical servers, VMs or cloud instances provide the actual compute, storage, network and hardware devices.
- Linux runs on top of this infrastructure.

Examples



Filesystem

(Files, directories, storage)



Network

(Sockets, TCP/IP, routing, firewall)



Devices

(Drivers, hardware interfaces)



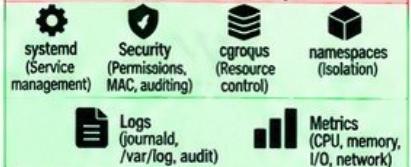
Hardware / Virtual Infrastructure

(Physical servers, VMs, cloud, storage, network)

8. OPERATIONAL OVERLAYS

- These layers work across the entire system and provide control, isolation, security, monitoring and observability.
- They are not separate layers but span multiple parts of the stack.

Key Operational Overlays



OPERATIONAL OVERLAYS (SPAN ACROSS ALL LAYERS)



systemd
(Service management, services, units, boot)



Security
(DAC, capabilities, SELinux/AppArmor, audit)



cgroups
(Resource limits, CPU, memory, I/O)



namespaces
(PID, network, mount, user, UTS)



Logs
(journal, /var/log, application logs, audit)



Metrics
(CPU, memory, I/O, network, latency, errors)

9. HOW TO INVESTIGATE (SYSTEM VIEW)

- Start from the symptom and understand the blast radius.
- Move through each layer and collect evidence.
- Use logs, metrics and system state to confirm or eliminate hypotheses.
- Look for recent changes and correlate across layers.
- Identify the root cause, mitigate, verify and prevent recurrence.

10. SENIOR INVESTIGATION PRINCIPLE



Do not ask, "Which command should I run?"
Ask, "Which subsystem could produce this symptom, and what evidence would prove or eliminate that hypothesis?"

11. KEY TAKEAWAYS

- Everything in Linux is connected.
- A symptom at the application layer can be caused by issues in CPU, memory, I/O, network, filesystems, security or infrastructure.
- Use a structured, systems-level approach.
- Correlate logs, metrics and recent changes.
- Understand the whole system, not just one layer.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta